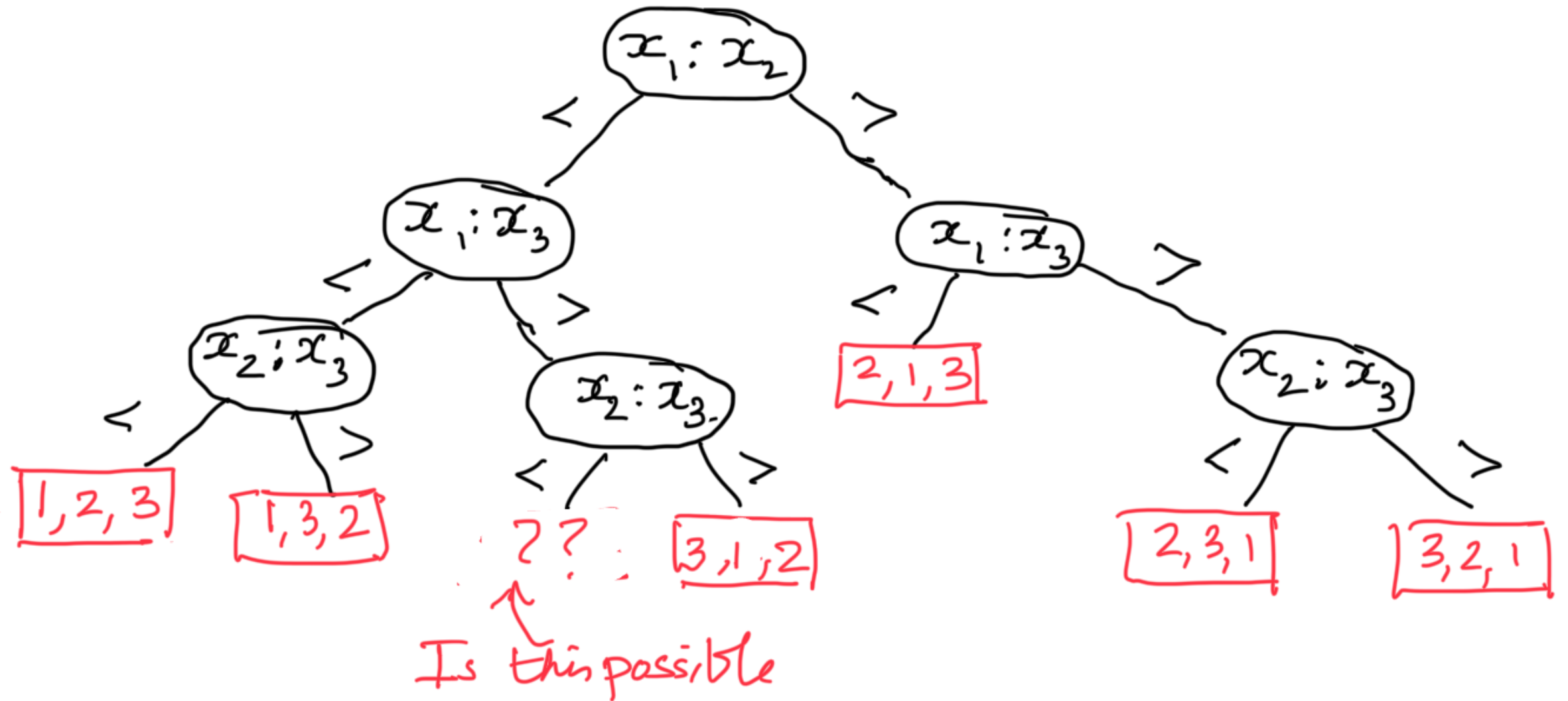
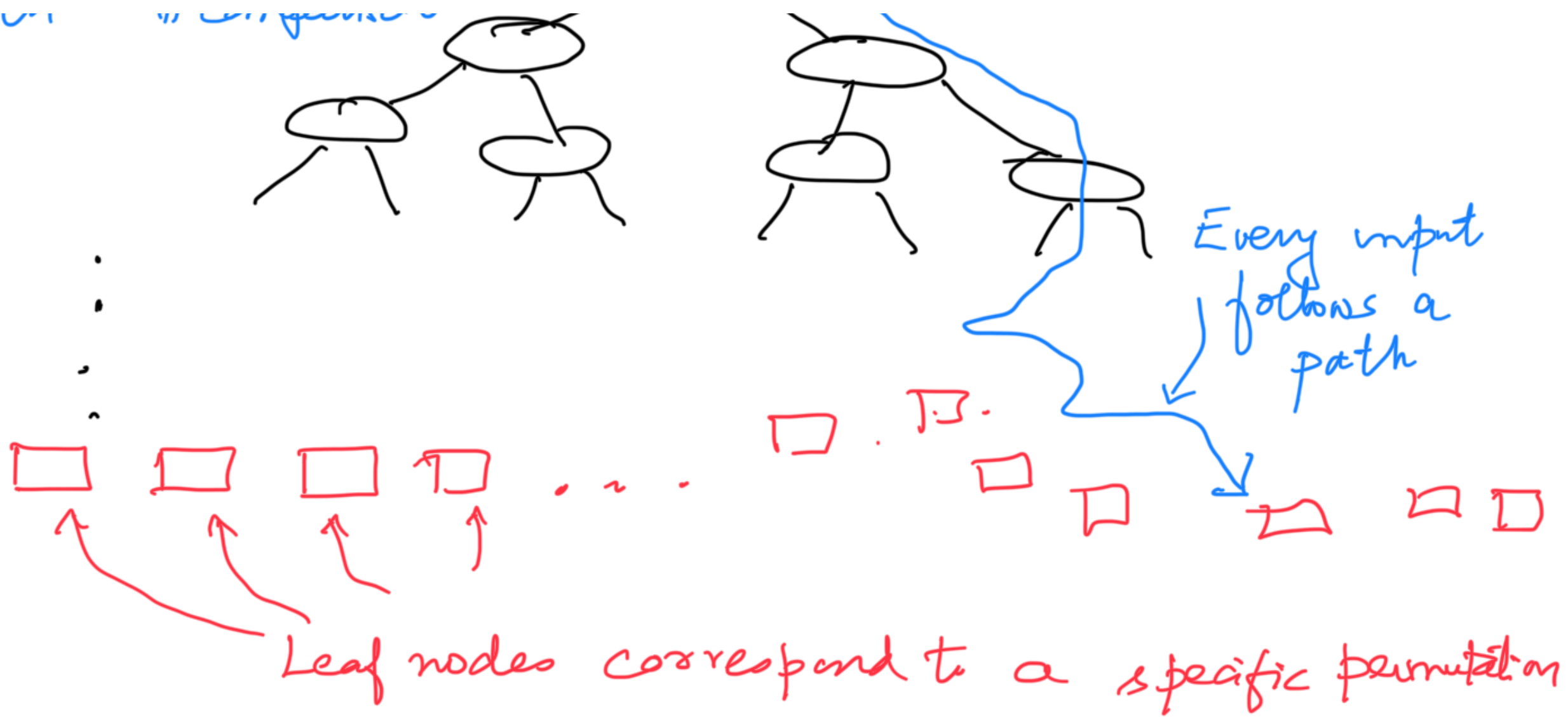


Complexity of comparison based sorting
Given x_1, x_2, x_3



Any sorting algorithm is a Comparison Tree for a fixed input size

The length of a path = # comparisons



Observation: Two inputs I_1^n, I_2^n will follow the same path iff they have the same ordertype

say $5, 2, 6, 10$ &
 $26, 3, 54, 80$

The ^{worst case} complexity of any comparison

tree $\hat{=}$ max path length from (root $\rightarrow$ leaf)

min
over all comparison
Trees $T \in \mathcal{T}$
inputs of length n

$\{ \text{max path length in } T \}$

$=$ lower bound of comparison
sorting

$$= \Omega(g(n))$$

Claim : $g(n) = \log(n!) = \Omega(\log(n^n))$
 $= \Omega(n \log n)$

Proof : Any (correct) comparison tree
has $\geq n!$ leaf nodes

$\Rightarrow$ the ht of any binary tree

with L leaf nodes has height $\geq \log L$

What is average lower bound (as opposed to worst case lower bound)

Assuming an input distribution on the ^{expected} permutations, what is the min length of the longest path $= \sum_{i \in \text{all leaves}} p_i d_i$
 p_i : prob of i^{th} leaf d_i : depth of i^{th} leaf

Suppose input distribution is uniform (all leaf nodes equally likely) p_i 's are same

Lower bound for average case = lower bound of the average path length over

all binary trees

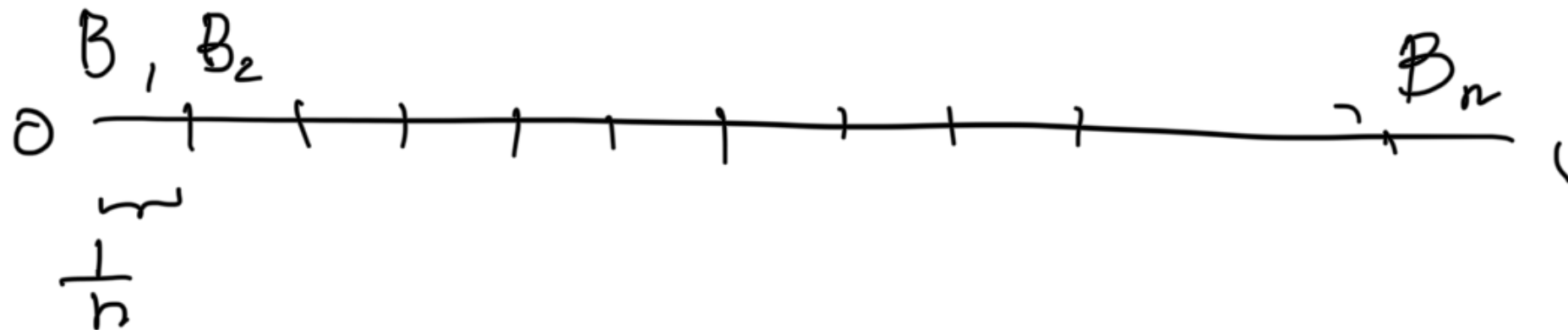
Sorting uniformly distributed numbers in $[0, 1]$

Given n numbers from
uniform distribution over $[0, 1]$

we want to sort them

$\{x_1, x_2, \dots, x_n\}$

- Create n buckets of $[0, 1]$



- For any number x_i , we put this
in the bucket $\lceil x_i \times n \rceil$

- Sort the elements in each bucket using some simple algorithm
- Output the sorted buckets $B_1, B_2, \dots, B_n$

Analysis of the expected running-time

- 
 1st step : $O(1)$ for each element
 2nd step : $\sum_i O(|B_i|^2)$ using an $O(n^2)$ algorithm
 3rd step : $O(n)$ to link the sorted buckets (leave out empty buckets)

$$E[|B_i|] = 1 \quad \text{from the observation}$$

that $P_r (x_j \in B_i) = \frac{1}{n}$

Fix a bucket B_k (will be same for all k)

Indicator
r.v.

$$X_i = \begin{cases} 1 & \text{if } x_i \in B_k \\ 0 & \text{otherwise} \end{cases}$$

$$\Rightarrow E[X_i] = \frac{1}{n} \quad \text{since } P_r(X_i=1) = \frac{1}{n}$$

$$\Rightarrow E[|B_k|] : E\left[\sum X_i\right] = \sum_{i=1}^n E[X_i] = 1$$

Number of comparisons to sort a bucket B_k

$$= |B_k|^2$$

$$E[|B_k|^2] = E\left[\left(\sum X_i\right)^2\right]$$

$$= E\left[\sum X_i^2 + 2 \sum_{i < j} X_i X_j\right]$$

$$\begin{aligned}
 &= \sum E[X_i^2] + 2 \underbrace{E\left[\sum_{i \neq j} X_i X_j\right]}_{2 \cdot \sum E[X_i \cdot X_j]} \\
 &= 2 \cdot \sum_{i \neq j} E[X_i] E[X_j] \\
 &\quad \text{since } X_i, X_j \text{ are ind.} \\
 &= \sum E[X_i^2] + 2 \sum_{i \neq j} E[X_i] \cdot E[X_j]
 \end{aligned}$$

Calculate this value exactly

and Total expected time to sort

$$= n \times \underline{\text{value of the exp}}$$

Suppose we are given n numbers that are integers from 1 to m

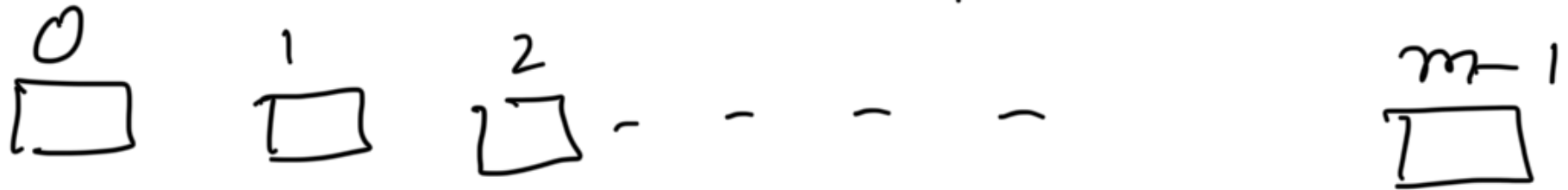
Eg.

$$m = 10$$

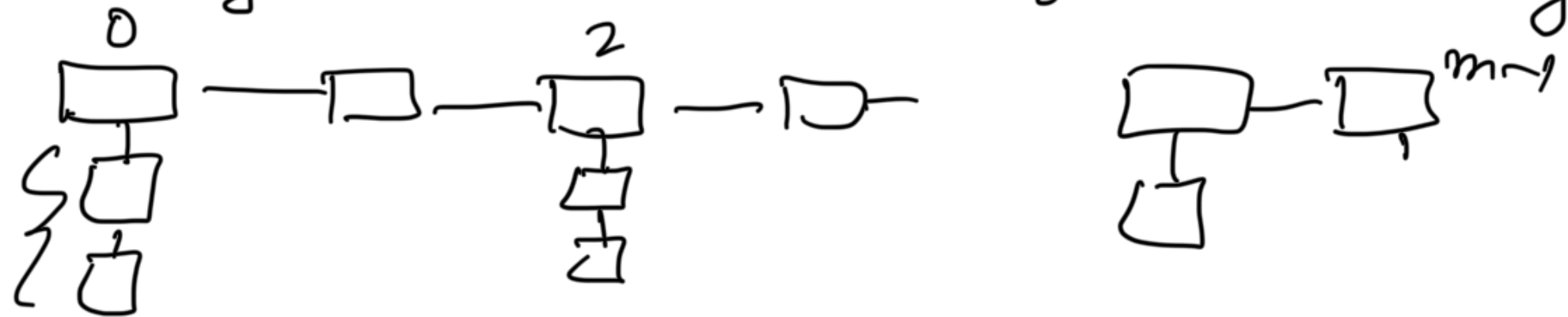
$$n = 20$$

3, 8, 4, 9, 6, 5, 4, ...

How should we sort?



Scan the numbers $x_1, x_2, \dots, x_n$ and put x_j in the bucket B_i where $i = x_j$.



Concatenate the buckets in order $B_0 - B_1 - \dots$
(non-embly)

Time = ? $O(n+m)$

Case : $n \sim m$ $O(n)$

$m \ll n$ $O(n)$

$n \ll m$ $O(m)$

Radix sort : It is used to increase

the range from $[0 \dots n-1]$

to $[0 \dots 10^d - 1]$
without sacrificing time